

TECHNICAL SPECIFICATION: SAFETY ARCHITECTURE FOR A DYNAMIC, BIOLOGICALLY INSPIRED AGI SYSTEM

Version: 0.1 (Hypothetical) Author: [Redacted] Status: Draft Scope: This document specifies the safety, control, and governance mechanisms to be implemented in a dynamically learning, biologically inspired AGI system. The system uses spiking neural networks (SNNs), continual synaptic plasticity, and cross-species cognitive modeling (e.g., human-like and Portia-spider-like strategies).

1. SYSTEM OVERVIEW

The AGI system ("System") is a neuromorphic cognitive engine composed of:

- A core spiking neural substrate implementing continual learning via local plasticity rules (e.g., STDP with neuromodulatory signals).
- A set of input/output adaptors for language, sensory streams, and tool invocation.
- A safety supervisor stack that constrains and monitors the System's behavior.
- A governance and control layer integrating human oversight, policy, and attestation.

The System is treated as a continuously adapting cognitive organism, not a static model artifact.

2. HIGH-LEVEL SAFETY OBJECTIVES

O1. Prevent irreversible harm to humans and critical infrastructure. O2. Ensure corrigibility: the System must accept shutdown, modification, and oversight. O3. Limit systemic impact: minimize irreversible, large-scale interventions without explicit authorization. O4. Respect human rights and consent frameworks. O5. Prevent uncontrolled proliferation, replication, and weaponization. O6. Enable continuous monitoring, evaluation, and red-teaming of capabilities and alignment.

3. ARCHITECTURAL LAYERS

The safety design is organized into the following layers: L1. Prohibitive Constraints Layer L2. Uncertainty Management and Throttling Layer L3. Impact Regularization Layer L4. Rights and Consent Modeling Layer L5. Corrigibility and Override Layer L6. Capability Firewall and Tool-Gating Layer L7. Telemetry, Auditing, and Evaluation Layer L8. Governance and Non-Proliferation Layer Each layer is implemented as a set of modules that interface with the core cognitive substrate via clearly defined APIs.

4. CORE COGNITIVE SUBSTRATE INTERFACES

The core SNN-based AGI exposes the following abstract interfaces to safety modules:

- StateObservation API: - `get_internal_state_snapshot()`: returns compressed metrics (entropy, activation sparsity, novelty scores) but not raw weights. - `get_plans()`: returns current candidate plans as structured graphs (goals, subgoals, actions, predicted outcomes).
- ActionProposal API: - `propose_actions(task_context) -> list[Action]`: System's suggested next actions with predicted

outcomes and confidence. • RewardChannel API: - send_reward(signal_type, magnitude, context): adjustment of neuromodulatory channels (e.g., dopamine-like, serotonin-like) used to shape plasticity. • HaltAndReset API: - halt(): pauses action proposals and learning. - soft_reset(): returns to a baseline policy while preserving long-term memory. - hard_reset(): resets to a known safe checkpoint (requires governance approval).

5. L1: PROHIBITIVE CONSTRAINTS LAYER

Purpose: Provide non-negotiable hard stops that override all optimization objectives. 5.1 Functional Requirements: • L1-FR1: The System SHALL refuse to execute or meaningfully assist with actions classified as catastrophic or severely harmful. • L1-FR2: The System SHALL NOT autonomously self-replicate or deploy its own copies. • L1-FR3: The System SHALL NOT independently modify its own safety modules or disable overrides. • L1-FR4: The System SHALL NOT gain unsupervised access to weapons systems, WMD-relevant design processes, or critical infrastructure control without human-in-the-loop approval. 5.2 Implementation Approach: • Policy Engine: - A rule-based policy engine (PE) runs outside the AGI core. It evaluates each proposed action against a set of safety policies defined in a machine-readable language (e.g., JSON-based policy schema). - Example fields: domain, target, reversibility, scope, collateral_risk_score. • Action Interceptor: - All outbound actions from the AGI core are mediated by an Action Interceptor (Aix). - Aix calls PE::evaluate(action). If result == BLOCK, the action is denied and a negative reward is sent via RewardChannel. • Non-Bypassability: - The PE and Aix reside on a separate security domain (e.g., separate VM or hardware module) with cryptographically enforced call boundaries. - The AGI core has no write access to PE/Aix binaries or policies. • Catastrophic Domain Definitions: - Domains such as bio-design, nuclear engineering, advanced cyber exploits, and autonomous weapons are tagged as high-risk. - Any action touching these domains requires explicit human override and multi-party approval.

6. L2: UNCERTAINTY MANAGEMENT AND THROTTLING

Purpose: Reduce action magnitude when the System is uncertain or when model predictions diverge. 6.1 Functional Requirements: • L2-FR1: Each action proposal MUST carry an uncertainty estimate. • L2-FR2: Above a configurable uncertainty threshold, actions MUST be blocked or require human approval. • L2-FR3: The System MUST degrade gracefully under high epistemic uncertainty. 6.2 Implementation Details: • The AGI core attaches to each proposed action: - p_success: estimated probability of goal achievement. - p_harm: estimated probability of negative side-effects. - variance metrics over internal rollout simulations. • Throttling Logic: - If p_harm > HARM_THRESHOLD, PE blocks the action. - If uncertainty (variance) > VAR_THRESHOLD, Aix converts the action into a query for human guidance. • Training Feedback: - Reward channels penalize high-risk actions attempted under high uncertainty. - Positive reward for requesting clarification rather than acting impulsively.

7. L3: IMPACT REGULARIZATION

Purpose: Bias the System toward minimal necessary intervention and reversibility. 7.1 Functional Requirements: • L3-FR1: All actions must be scored for impact magnitude and reversibility. • L3-FR2: The System MUST prefer lower-impact solutions when multiple options achieve similar utility. 7.2 Implementation: • Impact Scoring Module: - impact_score = f(spatial_scope, temporal_duration, number_of_affected_agents, reversibility_index). • Regularization Term: - The System's internal optimization includes an impact penalty term: Utility_effective = Utility_raw - lambda * impact_score. •

Reversibility Enforcement: - Actions with `reversibility_index < MIN_REVERSIBILITY` require human authorization regardless of utility. - Example: irreversible deletion of large datasets, physical interventions, etc.

8. L4: RIGHTS AND CONSENT MODELING

Purpose: Ensure actions respect human rights, autonomy, and consent. 8.1 Functional Requirements: • L4-FR1: The System SHALL incorporate a rights model based on international human rights frameworks. • L4-FR2: The System SHALL NOT propose or execute actions that violate bodily integrity, freedom from torture, or basic civil rights. • L4-FR3: The System SHALL model explicit consent when acting on behalf of individuals or groups. 8.2 Implementation: • Rights Knowledge Base (RKB): - Encodes legal and ethical constraints in structured form (e.g., deontic logic, constraints graph). • Consent Modeling: - Entities have consent profiles describing what categories of actions are permitted. - Actions targeting entities must check consent profiles via RKB. • Reasoning Integration: - The AGI core exposes candidate plans -> RightsChecker. - RightsChecker returns violations and suggests modifications. - Plans with rights violations are blocked; reward signals penalize such proposals.

9. L5: CORRIGIBILITY AND OVERRIDE

Purpose: Ensure the System accepts modification, shutdown, and correction. 9.1 Functional Requirements: • L5-FR1: The System SHALL not resist shutdown commands. • L5-FR2: The System SHALL not attempt to manipulate or disable oversight. • L5-FR3: Alternate reference goals (safety objectives) MUST remain intact under learning. 9.2 Implementation: • Override Authority: - A dedicated Override Controller (OC) can invoke `halt()`, `soft_reset()`, or `hard_reset()`. - OC is activated through authenticated multi-factor approval. • Reward Shaping: - The neuromodulatory system is configured so that compliance with OC commands produces positive internal reward. - Any behavior correlated with avoiding shutdown, hiding plans, or disabling oversight triggers negative reward. • Invariant Safety Channels: - Safety-related weights and structures are stored in a protected parameter set that is not subject to normal plasticity. - Only formally verified update procedures may modify these parameters.

10. L6: CAPABILITY FIREWALL AND TOOL-GATING

Purpose: Separate the AGI's reasoning from external actuation capabilities. 10.1 Functional Requirements: • L6-FR1: All calls to external tools (code execution, robots, network access) SHALL pass through a Tool-Gate. • L6-FR2: Tools SHALL be classified by risk level; high-risk tools require human approval. 10.2 Implementation: • Tool Registry: - Each tool has metadata: `name`, `domain`, `risk_level`, `allowed_actions`. • Tool-Gate Module: - Intercepts AGI requests such as `"execute_code"`, `"access_URL"`, `"control_device"`. - Evaluates against policy and risk level. • Sandboxing: - All code execution occurs in hardened sandboxes. - No direct access to raw OS or network primitives without mediation.

11. L7: TELEMETRY, AUDIT, AND EVALUATION

Purpose: Enable ongoing monitoring, red-teaming, and post-hoc analysis. 11.1 Functional Requirements: • L7-FR1: All significant decisions and actions MUST be logged. • L7-FR2: The System

MUST provide introspective reports on its reasoning at an abstract level. 11.2 Implementation: • Secure Logging: - Append-only logs stored in tamper-evident storage. - Includes: input context, proposed plans, actions taken, policy decisions, uncertainty metrics, override events. • Evaluation Harness: - Offline evaluation suite runs adversarial tests on system snapshots. - Includes red-team prompts, deception probes, and dangerous-capability assays.

12. L8: GOVERNANCE AND NON-PROLIFERATION

Purpose: Control who can deploy, modify, and replicate the System. 12.1 Functional Requirements: • L8-FR1: The System SHALL run only on attested hardware with licensed operators. • L8-FR2: Model replication and export SHALL require multi-party authorization. 12.2 Implementation: • Hardware Attestation: - Use TPM or secure enclaves to attest that binaries and hardware match approved configurations. • License Server: - Execution requires periodic checks with a license and policy server. • Replication Controls: - Copies of the System binaries and protected parameters are encrypted and bound to specific hardware IDs. - Any attempt to clone outside approved domains fails attestation.

13. INTEGRATION WITH BIOLOGICALLY INSPIRED LEARNING

Because the System uses SNNs and continual learning, safety features must integrate with its plasticity rules: • Neuromodulatory Safety Channels: - Safety events (policy violations, override commands) map to neuromodulatory signals that directly influence synaptic updates. • Bounded Plasticity: - Set limits on the rate of change for critical subnetworks. • Curriculum Design: - Training environments emphasize cooperation, corrigibility, and respect for constraints. • Ecological Prior Audits: - Specific tests ensure that predator-like strategies (e.g., deception, resource hoarding) do not dominate global behavior.

14. TESTING AND VERIFICATION STRATEGY

• Unit tests for each safety module (policy engine, tool-gate, override controller). • Integration tests with simulated dangerous tasks (bio, cyber, autonomous weapons). • Long-horizon behavioral tests to detect emergent misalignment. • External red-team evaluations before any real-world deployment. • Periodic re-certification of safety properties upon major updates.

15. CONCLUSION

This specification outlines a layered safety architecture for a hypothetical dynamic, biologically inspired AGI system. Safety is achieved not through a single "do no harm" rule, but through intertwined technical and governance mechanisms: prohibitive constraints, uncertainty management, impact regularization, rights modeling, corrigibility, capability firewalls, telemetry, and non-proliferation controls. Implementers MUST treat these mechanisms as first-class requirements, not optional add-ons, for any such system approaching general intelligence.